

**University Of Engineering and Technology, Lahore**  
**Computer Engineering Department**

|                                                                   |                                              |
|-------------------------------------------------------------------|----------------------------------------------|
| <b>Course Name:</b> Fundamentals of Programming & Data Science    | <b>Course Code:</b> CMPE-112L                |
| <b>Assignment Type:</b> Lab Report                                | <b>Dated:</b> July 1, 2026                   |
| <b>Semester:</b> 1st Semester                                     | <b>Session:</b> Spring 2026                  |
| <b>Lab/Project/Assignment #:</b> 14                               | <b>CLOs to be covered:</b> CLO2              |
| <b>Lab Title:</b> OS Library, if __name__ == "__main__", is vs == | <b>Teacher Name:</b> Hafiz Muhammad Mubashar |
| <b>Student Name / Roll No:</b> Hashir Zahid (2026(S)-CYS-40)      |                                              |

**Lab Evaluation:**

|              |                                                                                                 |               |               |               |               |               |
|--------------|-------------------------------------------------------------------------------------------------|---------------|---------------|---------------|---------------|---------------|
| <b>CLO2</b>  | Design and implement modular programs using functions, recursion, and reusable code constructs. |               |               |               |               |               |
|              | <b>Level1</b>                                                                                   | <b>Level2</b> | <b>Level3</b> | <b>Level4</b> | <b>Level5</b> | <b>Level6</b> |
| (5)          |                                                                                                 |               |               |               |               |               |
| <b>Total</b> |                                                                                                 |               |               |               |               | /5            |

**Rubrics for Current Lab Evaluation**

| Scale | Marks | Level | Rubric |
|-------|-------|-------|--------|
|-------|-------|-------|--------|

|                   |   |    |                                                               |
|-------------------|---|----|---------------------------------------------------------------|
| Excellent         | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good         | 4 | L2 | Submitted the lab tasks but have good understanding           |
| Good              | 3 | L3 | Submitted the lab tasks but have weak understanding.          |
| Basic             | 2 | L4 | Submitted the lab tasks but have no understanding.            |
| Barely Acceptable | 1 | L5 | Submitted only one lab task.                                  |
| Not Acceptable    | 0 | L6 | Did not attempt                                               |

## **LAB No 14**

### **Lab Goals / Objectives:**

- Understand the OS module and its functions for system operations
- Learn file and directory management using the os module
- Understand the purpose of if `__name__ == '__main__'` construct
- Differentiate between the 'is' operator and '==' operator
- Learn to create and manage modules in Python
- Understand Python program execution flow
- Apply OS operations for practical file management tasks

### **Equipment Required:**

- Computer System with Python 3.x installed
- Text Editor or IDE (PyCharm, VS Code, or IDLE)
- Operating System: Windows / Linux / macOS

## Theory:

### 1. The OS Module

The `os` module in Python provides functions for interacting with the operating system. It allows you to perform file and directory operations, manage processes, and access environment variables. The `os` module is part of Python's standard library and does not require installation.

### 2. Important OS Module Functions

- `os.getcwd()` - Returns the current working directory as a string
- `os.chdir(path)` - Changes the current working directory to the specified path
- `os.listdir(path)` - Returns a list of all files and directories in the specified path
- `os.mkdir(path)` - Creates a new directory at the specified path
- `os.makedirs(path)` - Creates a directory and all intermediate-level directories
- `os.rmdir(path)` - Removes an empty directory
- `os.remove(path)` - Deletes a file at the specified path
- `os.rename(src, dst)` - Renames a file or directory from `src` to `dst`
- `os.path.exists(path)` - Returns `True` if the path exists
- `os.path.isfile(path)` - Returns `True` if the path is a file
- `os.path.isdir(path)` - Returns `True` if the path is a directory

### 3. Working with Directories

The `os` module provides comprehensive directory management capabilities. You can create, navigate, list, and remove directories programmatically:

```
import os
print(os.getcwd()) # Current directory
os.mkdir('newfolder') # Create directory
print(os.listdir('.')) # List contents
```

#### 4. `if __name__ == '__main__':`

This construct is a fundamental Python idiom. When a Python file is run directly, the special variable `__name__` is set to `'__main__'`. However, when the file is imported as a module, `__name__` is set to the module's name.

```
def main():  
    print('Running directly')  
  
if __name__ == '__main__':  
    main()
```

This allows code to behave differently when run as a script versus when imported as a module. It is the standard way to write reusable Python modules.

#### 5. Why Use `if __name__ == '__main__':`?

- Prevents code from running when the module is imported
- Allows the file to be both a reusable module and a standalone program
- Makes the code more organized and testable
- Follows Python best practices for module design

#### 6. The `'is'` Operator vs `'=='` Operator

The `==` operator compares the values of two objects for equality.

The `is` operator checks whether two references point to the same object in memory (identity comparison).

```
a = [1, 2, 3]  
b = [1, 2, 3]  
print(a == b) # True (same values)  
print(a is b) # False (different objects)
```

#### 7. When to Use `'is'`

The `is` operator is primarily used for comparing with singleton objects like `None`, `True`, and `False`:

```
if x is None:  
    print('x is None')
```

Using `is` for value comparison can lead to unexpected behavior because it checks identity, not equality.

## 8. Python Modules and Imports

A module is a Python file containing functions, classes, and variables that can be imported into other programs. The `import` statement makes a module's contents available:

```
import os  
import Hashir # Custom module  
Hashir.welcome()
```

## 9. String Methods: `isdigit()`, `isspace()`

- `str.isdigit()` - Returns `True` if all characters in the string are digits
- `str.isspace()` - Returns `True` if all characters are whitespace characters

These methods are useful for input validation and data cleaning.

## 10. The `globals()` Function Revisited

The `globals()` function returns a dictionary of the current global symbol table. It can be used to access global variables by their string names, which is useful for dynamic variable access.

## Lab Work:

### Task 1: Creating a Python Module

Create a custom Python module with functions and demonstrate importing it.

### Code Summary:

This task demonstrates creating a reusable Python module. The module defines two functions (welcome and Goodbye) and uses `if __name__ == '__main__':` to run test code only when the module is executed directly.

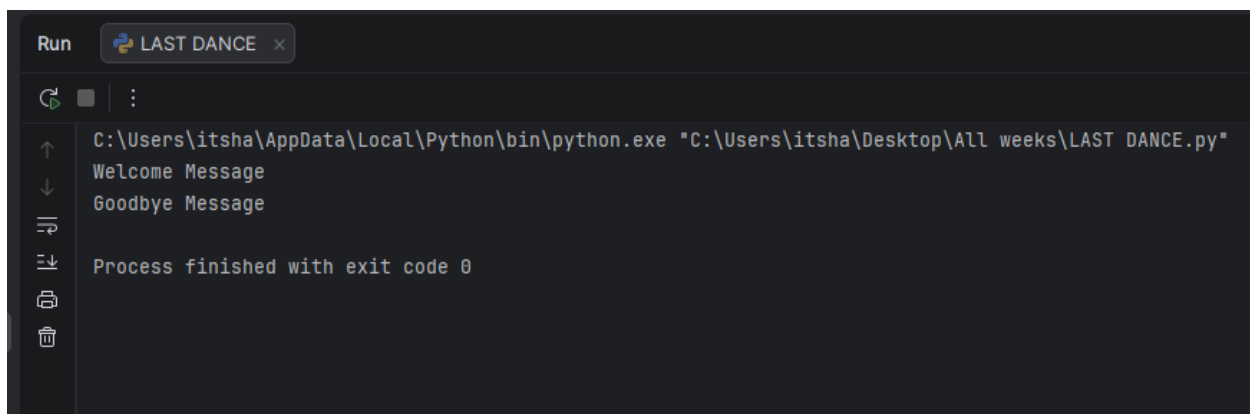
### Key Code Lines:

```
# Module: Hashir.py
def welcome():
    print("Welcome Message")

def Goodbye():
    print("Goodbye Message")

if __name__ == '__main__':
    welcome()
    Goodbye()
```

### Output:



```
Run  LAST DANCE  x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\LAST DANCE.py"
Welcome Message
Goodbye Message
Process finished with exit code 0
```

## Task 2: Importing a Custom Module

Write a program that imports and uses functions from a custom module.

### Code Summary:

This task shows how to import a custom module and call its functions. The module's functions are accessed using the module name as a prefix.

### Key Code Lines:

```
# Importing Custom Module
import Hashir
Hashir.welcome()
Hashir.Goodbye()
```

**Output:** No real Output required for this task according to sir

### Task 3: OS Commands Demonstration

Write a program that demonstrates five common OS module functions.

#### Code Summary:

This task demonstrates practical OS operations: getting the current directory, listing files, renaming files, deleting files, and removing directories.

#### Key Code Lines:

```
# Five OS Python Commands
import os

os.getcwd()          # Get current working directory
os.listdir()         # List all files in current directory
os.rename("old.py", "new.py") # Rename a file
os.remove("file.py") # Delete a file
os.rmdir("folder")   # Remove directory
```

**Output:** No real Output required for this task according to sir

### Task 4: Creating Multiple Directories

Write a program that creates multiple directories using a loop.

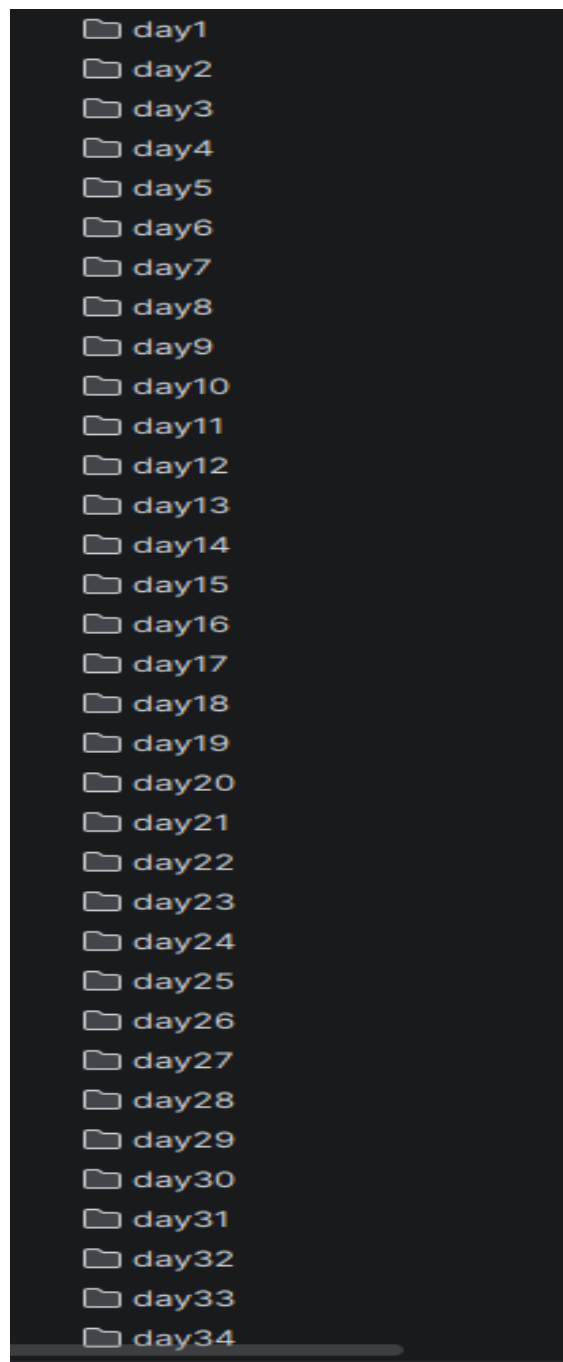
#### Code Summary:

This task demonstrates creating directories programmatically using `os.mkdir()` inside a loop, which is useful for batch directory creation.

#### Key Code Lines:

```
# Creating Multiple Directories
import os
os.mkdir("name")
for i in range(0, 100):
    os.mkdir(f"day{i+1}")
```

**Output:**



### **Task 5: is vs == Demonstration**

Write a program that demonstrates the difference between 'is' and '==' operators.

#### **Code Summary:**

This task clearly demonstrates that == compares values while is compares object identity (memory location). Two lists with the same values are equal but not identical.

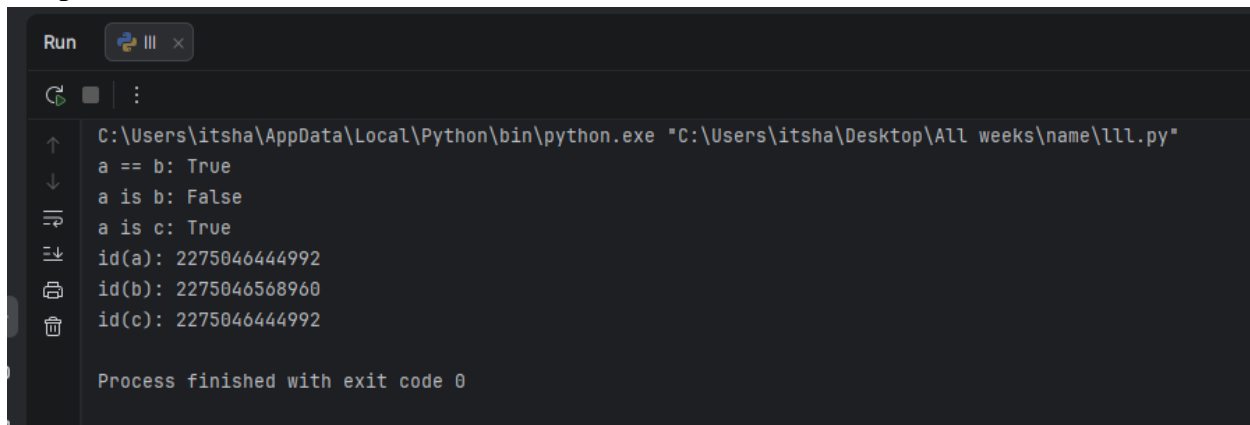


## Key Code Lines:

```
# is vs == Demonstration
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print("a == b:", a == b)    # True (same values)
print("a is b:", a is b)    # False (different objects)
print("a is c:", a is c)    # True (same object)
print("id(a):", id(a))
print("id(b):", id(b))
print("id(c):", id(c))
```

## Output:

A screenshot of a Python IDE terminal window. The window has a dark background with a light-colored border. At the top, there is a 'Run' button and a close button. Below the button, there is a command prompt icon and a list of icons. The main area of the terminal shows the output of the code: 'C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\l11.py"', 'a == b: True', 'a is b: False', 'a is c: True', 'id(a): 2275046444992', 'id(b): 2275046568960', 'id(c): 2275046444992', and 'Process finished with exit code 0'.

## Conclusions:

- Successfully understood and applied the OS module for system operations
- Learned to create, import, and use custom Python modules
- Understood the purpose and importance of if `__name__ == '__main__'`
- Differentiated between the 'is' operator (identity) and '==' operator (equality)
- Applied OS functions for file and directory management
- Understood Python program execution flow and module design
- Gained practical skills in writing reusable and well-structured Python code